

ミニゲーム提出マニュアル

Unity Package での書き出しと提出（全6項目）

【1】提出するもの

- 1. ミニゲームは、チームごとに別々のUnityプロジェクトで作られています。これを1つのプロジェクトにまとめるには、プロジェクトを丸ごと渡すのではなく、そのミニゲームを動かすのに必要なファイル一式を取り出して渡す必要があります。
- 2. そのための仕組みが、Unityに用意されている Unity Package です。

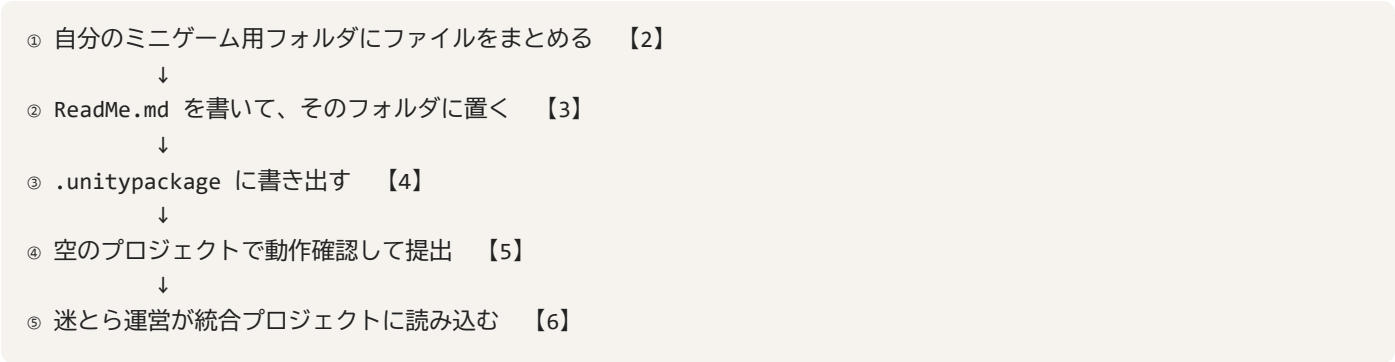
用語：Unity Package（.unitypackage）
Unityのプロジェクトから、指定したファイルをまとめて1つのファイルに書き出したもの。受け取った側は、それを自分のプロジェクトに読み込むと、フォルダ構成そのままファイルが追加されます。

- 3. 提出するものは2つです。

提出物	中身
.unitypackage ファイル	ミニゲームのファイル一式
ReadMe.md	ゲームの説明書（操作方法、追加した機能など）

※ReadMe.mdは、パッケージの中に含める形で提出します（作り方は【3】）。

- 4. 作業の流れはこうなります。



- 5. 【最重要】提出するファイルの名前は、全チーム統一します。

GameJam_<開始年yyyy>_<開始月mm>_Team<チーム名>_<ゲームタイトル>

例：

GameJam_2026_07_TeamA_MazeRunner.unitypackage
GameJam_2026_07_TeamB_ShootingStar.unitypackage
GameJam_2026_07_TeamC_RhythmCat.unitypackage

- `<>` は書きません。`Team` の後にそのままチーム名（A、B、C...）をつなげます
- 開始年月は、ゲームジャムが始まった年と月を2桁で書きます（2026年7月なら `2026_07`）
- 半角英数とアンダースコア（`_`）だけを使います。日本語やスペースは、読み込む側で不具合の原因になります

6. なぜ名前を統一するのか。迷とら運営の画面には、全チームのファイルが並びます。

○ 統一されている場合

```
GameJam_2026_07_TeamA_MazeRunner.unitypackage  
GameJam_2026_07_TeamB_ShootingStar.unitypackage  
GameJam_2026_07_TeamC_RhythmCat.unitypackage  
→ どれが誰のもので、いつの提出物か一目で分かる
```

× バラバラの場合

```
game.unitypackage  
最新版.unitypackage  
test2.unitypackage  
→ 中身を開くまで分からない。同名ファイルで上書き事故も起きる
```

【2】自分のフォルダにまとめる

1. 書き出しの前に、ファイルの置き場所を整理します。ここをやらないと、統合するときに全チームのファイルが混ざります。
2. よくない例から見てみます。作りながら進めると、だいたいAssetsフォルダはこうなっています。

```
Assets/  
├ Scenes/  
├ Scripts/  
├ Images/  
├ Prefabs/  
└ Audio/
```

一見きれいですが、**このまま書き出すと問題が起きます**。統合プロジェクトに全チーム分を読み込むと、こうなるからです。

```
Assets/  
├ Scenes/      ← 全チームのシーンが1か所に集まる  
├ Scripts/     ← 全チームのスク립トが混ざる  
├ Images/      ← 全チームの画像が混ざる  
└ ...
```

どれが誰のファイルか分からなくなりますし、**同じ名前のファイルがあると上書きされます**。`Player.cs` や `Title.unity` のような名前は、まず間違いなくかぶります。

3. そこで、**自分のミニゲーム専用のフォルダを1つ作って、その中に全部入れます**。

```
Assets/  
└ GameJam_2026_07_TeamA_MazeRunner/  
    ├── Scenes/  
    ├── Scripts/  
    ├── Images/  
    ├── Prefabs/  
    └ Audio/
```

こうしておけば、統合プロジェクトに読み込んでも、チームごとにきれいに分かります。

```
Assets/  
├ GameJam_2026_07_TeamA_MazeRunner/  
├ GameJam_2026_07_TeamB_ShootingStar/  
└ GameJam_2026_07_TeamC_RhythmCat/
```

4. フォルダ名は、提出ファイルと同じ名前にします。

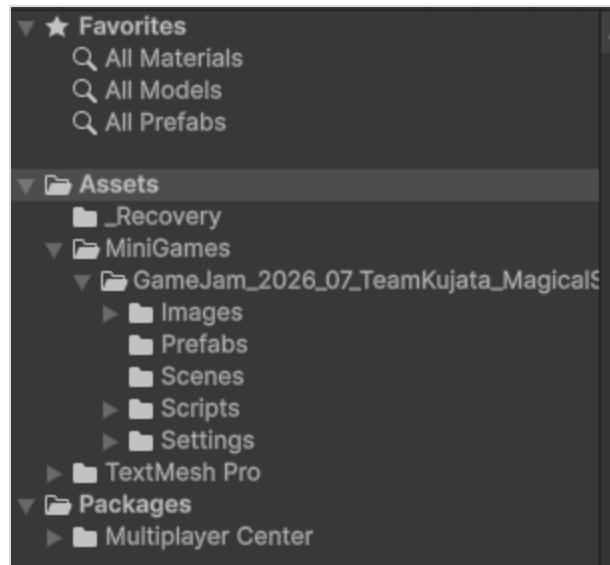
```
GameJam_<開始年yyyy>_<開始月mm>_Team<チーム名>_<ゲームタイトル>
```

ファイル名とフォルダ名がそろっていると、迷ったら運営が「このフォルダはどのチームのものか」を確認せずに済みます。

5. では作ります。UnityのProjectウィンドウで、Assetsフォルダを右クリック →「Create」→「Folder」。

名前を上形式（例： `GameJam_2026_07_TeamA_MazeRunner` ）にします。

6. 次に、既存のフォルダをこの中へ移動します。



Projectウィンドウでフォルダをまとめた状態

必ずUnityのProjectウィンドウ上でドラッグして移動してください。

エクスプローラー（Finder）で直接動かすと、Unityが参照を追えなくなって、シーンからスクリプトや画像が外れることがあります。

用語：.metaファイル

Unityが自動で作る、素材とセットの設定ファイル。**Unity上で移動すれば.metaも一緒に動く**ので参照は壊れませんが、エクスプローラーで動かすと.metaだけ取り残されて壊れる場合があります。フォルダの整理はUnity上で、が鉄則です。

7. 移動が終わったら、Unityを再生（Play）して、ゲームがちゃんと動くか確認します。

ここで動かなくなっていたら、参照が外れている可能性があります。**書き出す前に必ず直してください。**

8. この時点でよくあるトラブルはこの3つです。

症状	原因と対処
シーンを開いたらオブジェクトが白い四角になった	画像の参照が外れている。Inspectorで画像を割り当て直す
Missing (Mono Script) と表示される	スクリプトの参照が外れている。Inspectorでスクリプトを割り当て直す
Consoleに赤いエラーが出る	エラー内容を読んで対処。まず移動前の状態に戻せないか確認

【3】ReadMe.mdを書く

1. パッケージのファイルだけ渡されても、迷とら運営には「どう操作するのか」「どのシーンから始まるのか」「何を追加したのか」が分かりません。

そこで、説明書のファイルを一緒に提出します。名前は `ReadMe.md` に統一します。

用語：ReadMe（リードミー）

「最初に読んでね」という意味の、説明書ファイルの定番の名前。フォルダの中にReadMeがあれば、まずそれを読むという習慣があります。

用語：.md（マークダウン）

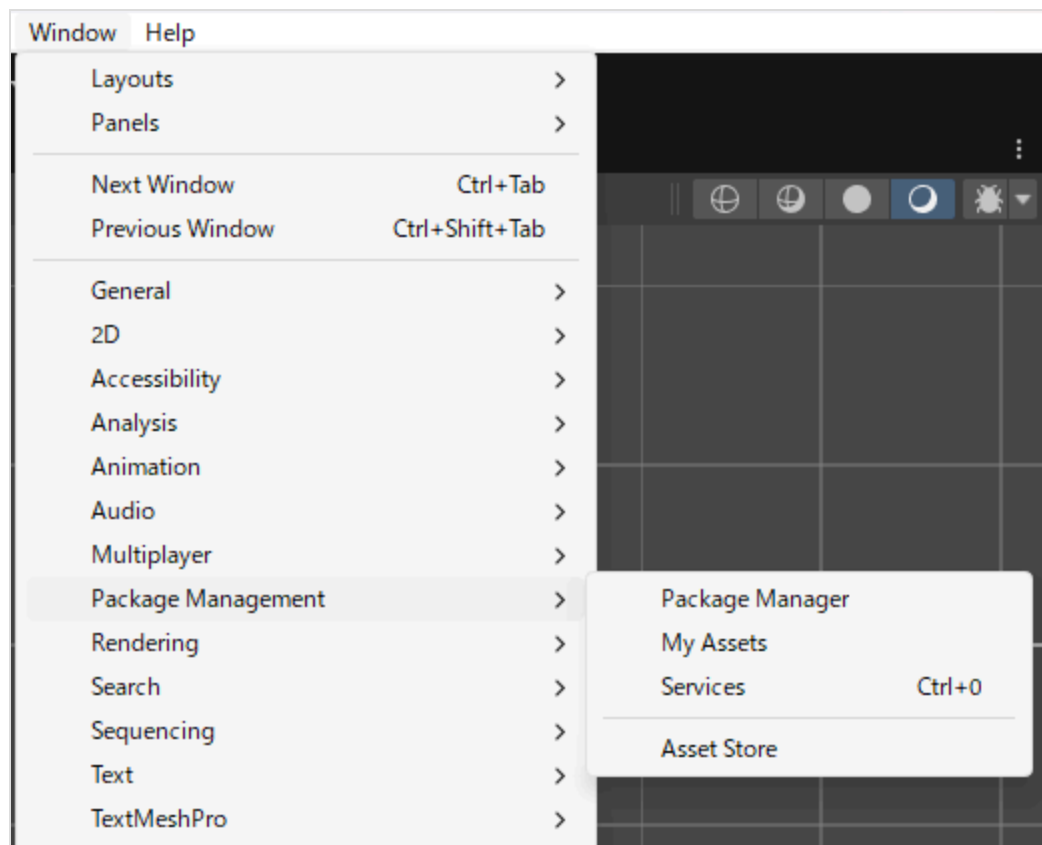
文字だけで見出しや箇条書きを書けるファイル形式。`#` を付ければ見出し、`-` を付ければ箇条書きになります。

2. まず、書く内容のうち調べないと分からない2つを先に確認します。

3. 【調べる1】Package Managerで追加した機能

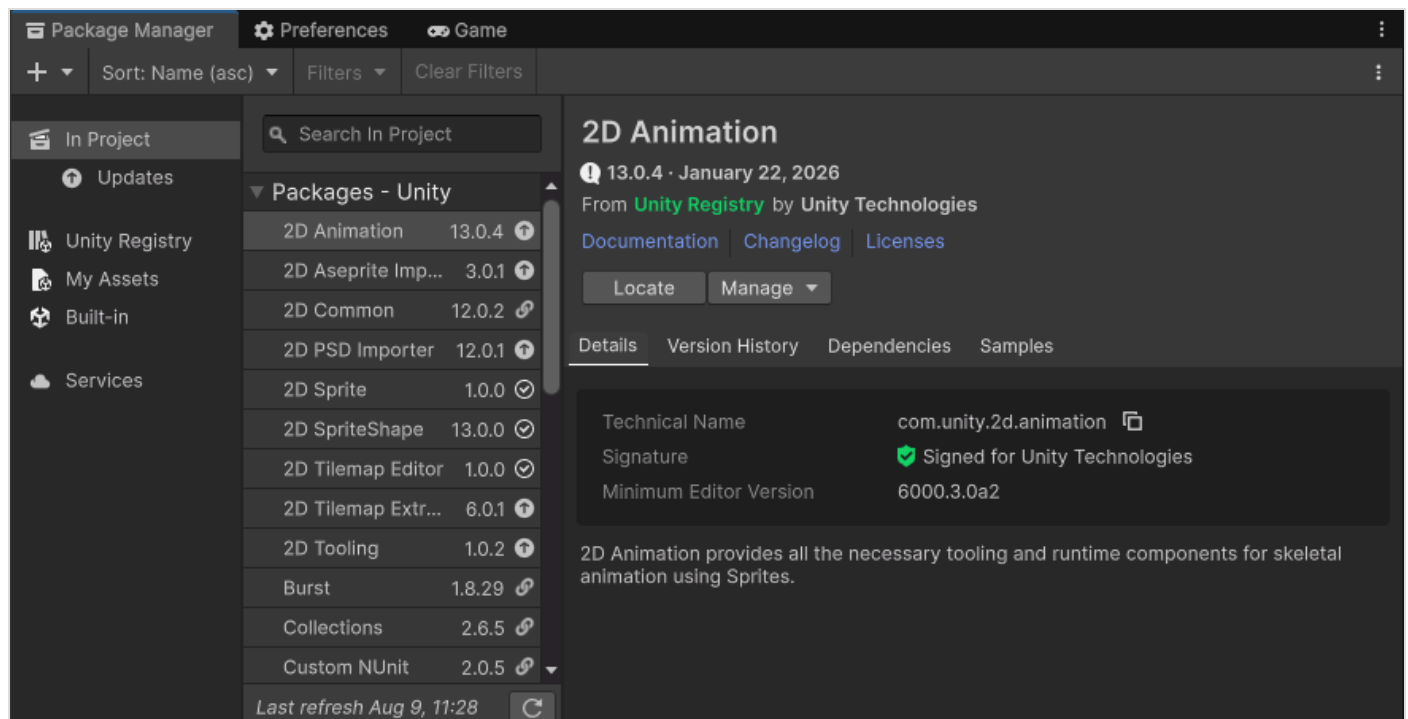
TextMeshPro、Cinemachine、DOTweenなど、後から追加した機能はパッケージに入りません。統合プロジェクトに同じものを入れてもらう必要があるので、名前を控えます。

`Window > Package Management > Package Manager` を開きます。



Package Managerを開くメニュー

左側の「In Project」を選ぶと、このプロジェクトで使っている機能の一覧が表示されます。



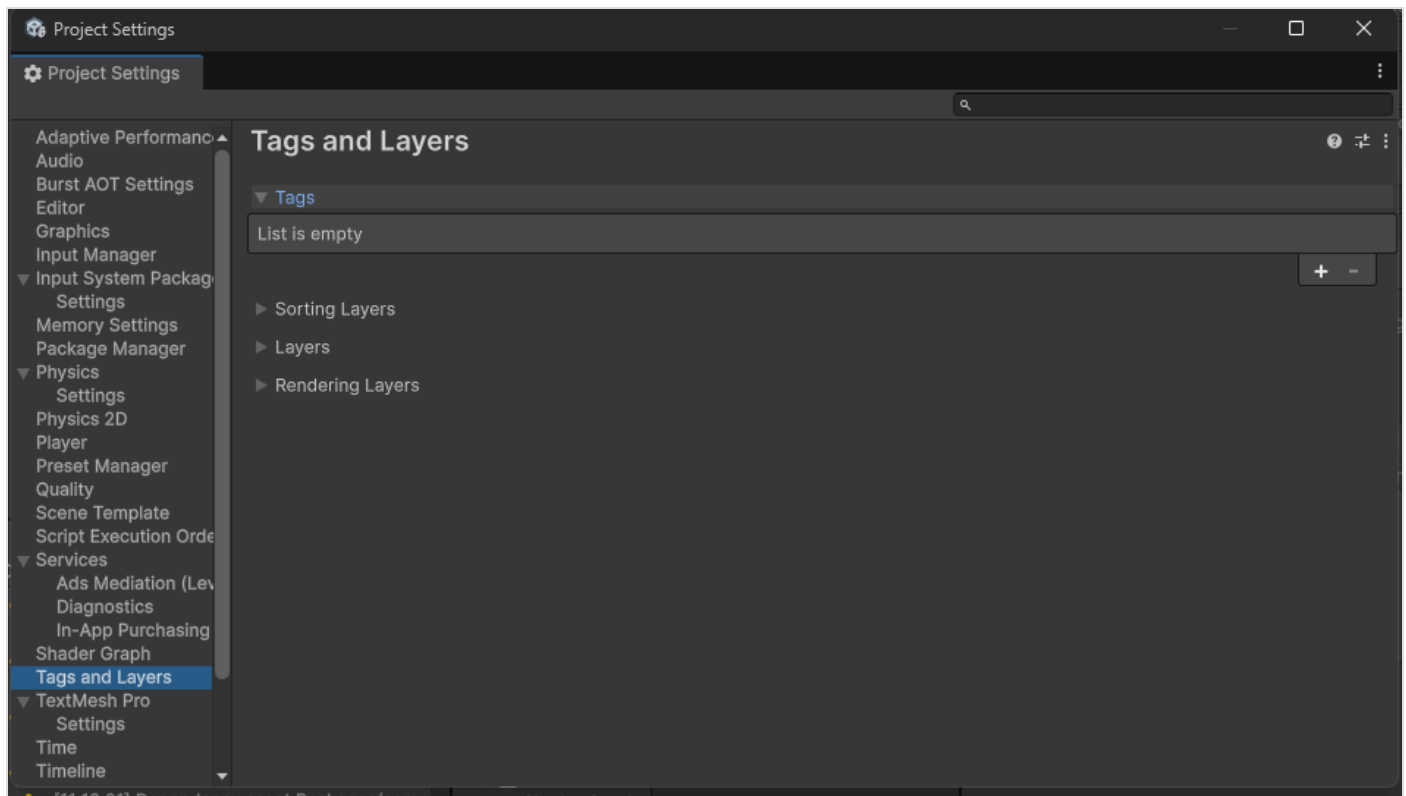
Package ManagerのIn Project表示

※Unityに最初から入っているものはそのままOKです。自分で追加した覚えのあるものだけ控えてください。

4. 【調べる2】自分で追加したタグ・レイヤー

Project Settings の中身は、パッケージには**一切入りません**。自分で追加したタグやレイヤーがあると、統合先では存在しないため、実行時にエラーで止まります。

Edit > Project Settings... を開いて、左のリストから「Tags and Layers」を選び、「Tags」の欄を開きます。



Tags and Layersの画面

※スクショのように「List is empty」と表示されていれば、自分で追加したタグは無いということです。この場合は「なし」と書きます。

※ Untagged MainCamera Player などの最初からあるタグは、統合先にも最初からあるので大丈夫です。

5. ここからReadMe.mdを作ります。使うのは **HackMD** というWebサービスです。

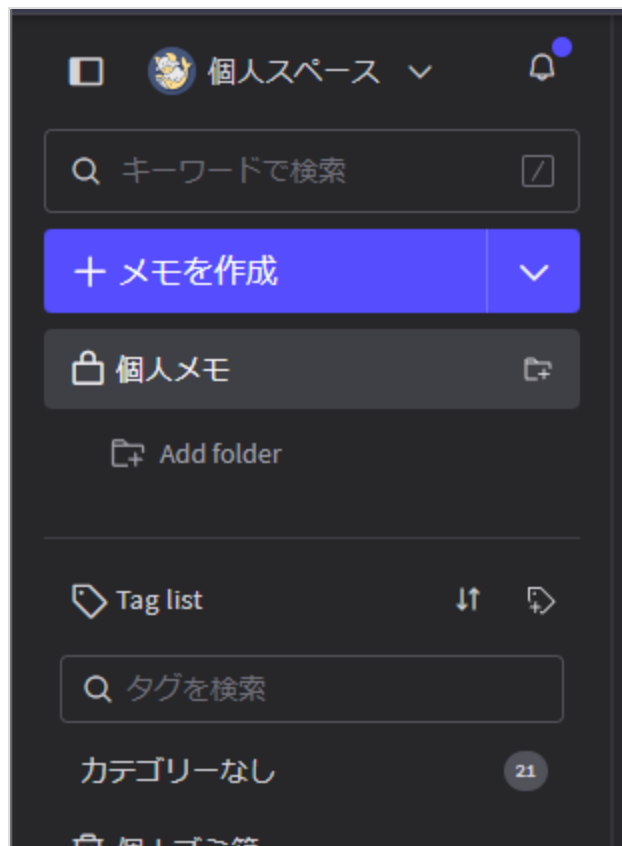
用語：HackMD

ブラウザ上でマークダウン（.md）を書けるサービス。左に書いた文字が、右にすぐ表示されて確認できます。**WindowsでもMacでも同じ画面**なので、メモ帳やテキストエディットの違いを気にせずに済みます。

<https://hackmd.io/>

※アカウント登録が必要です。GoogleアカウントやGitHubアカウントでログインできます。

6. ログインしたら、左上の「+ メモを作成」を押します。



HackMDのメモを作成ボタン

7. 空のメモが開くので、下のテンプレートをそのままコピーして貼り付け、中身を自分のゲームの内容に書き換えます。

```
# ゲームタイトル

## ゲーム概要
（どんなゲームか2～3行で。ジャンル、目的、勝敗条件など）

## 操作方法
- 移動：
- 決定：
- その他：

## 想定プレイ時間
（15秒～5分の範囲で。この時間でゲームが強制終了されます）

## 最初に開くシーン
（例：Title_TeamA.unity）

## 追加した機能（Package Manager）
- 無ければ「なし」

## 追加したタグ
- 無ければ「なし」

## 追加したレイヤー
- 無ければ「なし」

## その他
- 終了条件：（どうなったらゲームが終わるか）
```

- 使用素材：（配布サイトの名前とURL。自作なら「自作」）
- 既知の不具合：
- 制作メンバー：
- 連絡先：（DiscordのID）

8. 書くと、左に入力した内容が右にすぐ表示されます。見出しや箇条書きがきれいに表示されていれば、書き方は合っています。



HackMDの編集画面

9. 各項目が何に使われるかを知っておくと、書く内容に迷いません。

項目	迷とら運営がこう使う
ゲームタイトル	タイトル画面のメニューに表示する名前になる
ゲーム概要	遊び方が分からないときに読む。紹介文にも使う
操作方法	動作確認に必要。遊ぶ人への案内にも使う
想定プレイ時間	この時間を過ぎるとゲームが強制終了されます。最短15秒～最長5分の範囲で書いてください。実際の時間と大きくずれていると、遊びきる前に終了したり、次のミニゲームに進めなくなります
最初に開くシーン	タイトル画面からこのシーンへつながります。これが無いと組み込みません
追加した機能	統合プロジェクトに同じ機能を入れます。無いとエラーで動きません
追加したタグ	統合プロジェクトに同じタグを追加します。無いと実行時に止まります
追加したレイヤー	同上。当たり判定に関わるので、抜けると挙動が変わります
終了条件	ミニゲームが終わったらタイトルに戻す処理を入れるときに使います
使用素材	配布サイトによっては表記が必要です。まとめてクレジットに載せます

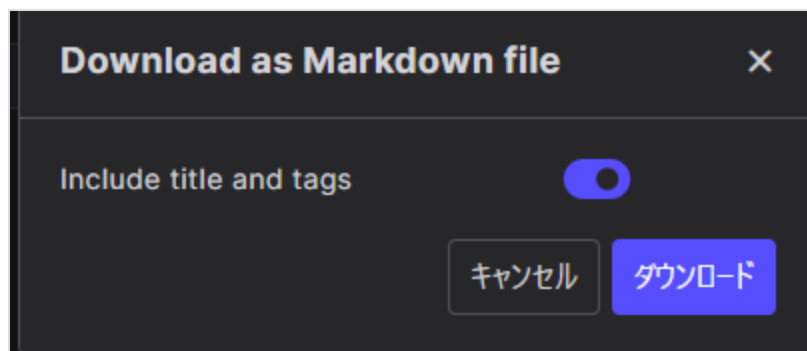
既知の不 具合	統合後に発覚したとき「元からか、統合で壊れたか」の判断材料になります
連絡先	統合中に不明点が出たとき、すぐ確認できます

10. 書いたら、ファイルとして書き出します。右上の「...」→「ダウンロード」→「Markdown」を選びます。



HackMDのダウンロードメニュー

11. 確認画面が出るので、そのまま「ダウンロード」を押します。



※「Include title and tags」をオンのままにすると、ファイルの先頭にタイトル情報の数行が追加されます。付いていても問題ありませんが、気になる場合はオフにしてください。

12. ダウンロードしたファイルは、**メモのタイトルがそのままファイル名**になっています（例： **迷路ダッシュ.md**）。

これを **ReadMe.md** に名前を変更してください。

13. 名前を変更したら、**自分のミニゲームのフォルダの中**に入れます。

```
Assets/  
└─ GameJam_2026_07_TeamA_MazeRunner/  
    │ ReadMe.md      ← ここに置く  
    │ Scenes/  
    │ Scripts/  
    └─ ...
```

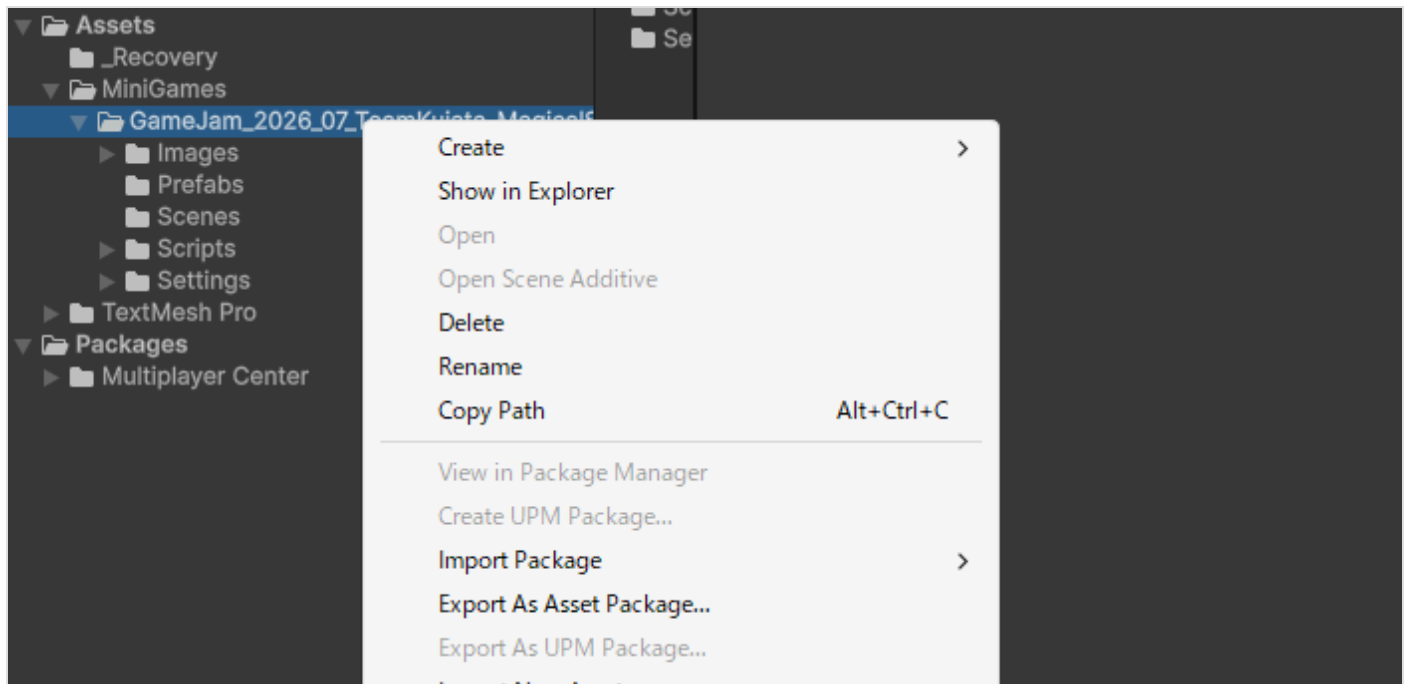
ここに置いてからパッケージを書き出すことで、ReadMe.mdもパッケージに含まれます。

14. 書くときの注意は3つです。

- 「なし」も書く。空欄だと「書き忘れ」なのか「無い」のか分かりません
- シーン名は**拡張子（.unity）まで正確に**。Title と title の違いでも動きません
- 想定プレイ時間は**15秒～5分**の範囲で。実際に遊んで測った時間を書きます

【4】パッケージに書き出す

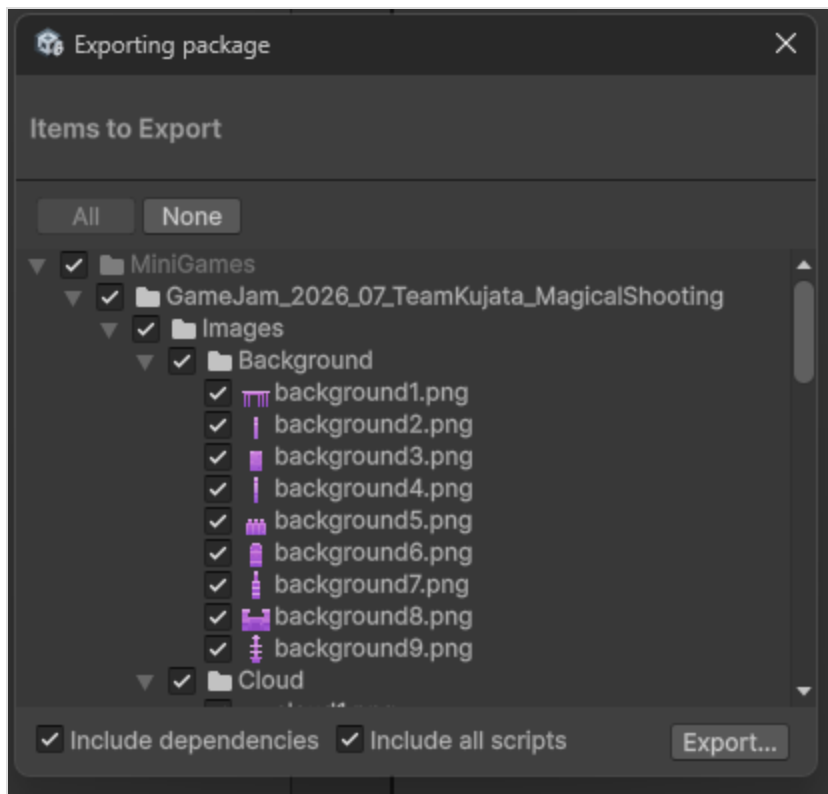
1. フォルダの整理とReadMe.mdの準備ができれば、書き出します。
2. Projectウィンドウで、自分のミニゲーム用フォルダ（ `GameJam_<開始年yyyy>_<開始月mm>_Team<チーム名>_<ゲームタイトル>` ）を右クリックします。
メニューの中から「Export As Asset Package...」を選びます。



自分のフォルダを右クリックしてExport As Asset Package

- ※右クリックするのは **Assets** ではありません。Assetsを右クリックすると、他のフォルダも含めた全部が対象になってしまいます。
- ※選んだフォルダの名前が青くハイライトされていることを確認してから、右クリックしてください。
- ※Unityのバージョンによっては「Export Package...」という名前です。どちらも同じ機能です。

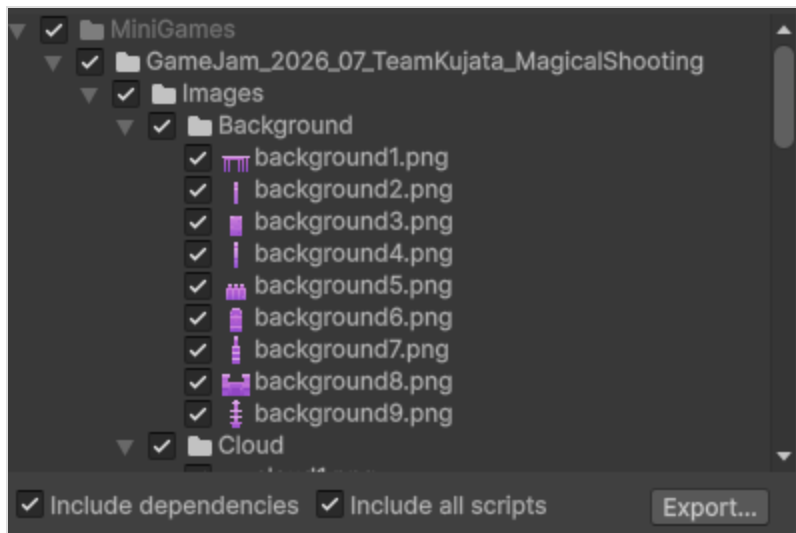
3. 「Exporting package」というウィンドウが開きます。ここが書き出しの設定画面です。



Exporting packageウィンドウ

4. 書き出す前に、次の2点を確認します。

確認1：自分のフォルダより下の階層すべてに、チェックが入っているか



自分のフォルダ以下に全部チェックが入っている状態

リストに表示されるのは、選んだフォルダとその中身だけです。すべての行にチェックが入っていれば正しい状態です。

【2】の手順どおりに操作していれば、ここは変更する必要はありません。チェックが外れている行があれば、クリックして入れてください。

確認2：下にある2つのチェックボックスが両方オンになっているか

- **Include dependencies**：選んだファイルが使っている他のファイルも自動で追加する
- **Include all scripts**：プロジェクト内のスクリプトをまとめて追加する

用語：Include dependencies（依存関係を含める）

例えばプレハブを1つ選んだとき、そのプレハブが使っている画像やスクリプトも一緒に書き出す設定です。オフにすると、統合先で「絵が付いていない」「スクリプトが無い」状態になります。特別な理由がなければ、オンのままにしてください。

5. 確認できたら、右下の「Export...」ボタンを押します。

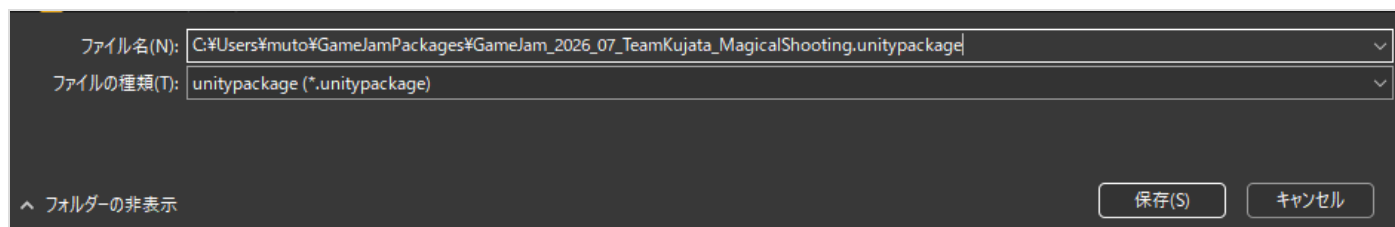
保存先を聞かれるので、分かりやすい場所（デスクトップなど）を選び、ファイル名を入力します。

6. ファイル名は、決められた形式で付けます。

GameJam_<開始年yyyy>_<開始月mm>_Team<チーム名>_<ゲームタイトル>

例：

GameJam_2026_07_TeamA_MazeRunner



保存ダイアログでのファイル名

※スクショは操作画面の例です。実際は自分のチーム名とゲームタイトルを入れてください。

※ **.unitypackage** の部分は自動で付くので、入力しなくて大丈夫です。

※半角英数とアンダースコア（_）だけを使ってください。

7. 「保存」を押すと書き出しが始まります。プロジェクトの大きさによっては1分ほどかかります。

指定した場所に **.unitypackage** ファイルができていれば成功です。

8. 最後に、ファイルサイズを確認します。

- **数MB～数十MB**：正常です
- **数百バイト～数KB**：ほとんど中身が入っていません。手順2でフォルダを選び忘れた可能性があります
- **1GB以上**：本来入らないものが混ざっている可能性があります。書き出す対象を見直してください

【5】提出前の最終確認

- 書き出したパッケージをそのまま提出する前に、動作確認をします。

自分のプロジェクトでは動くのに、統合先では動かないというのが、パッケージで一番多いトラブルだからです。

- 原因は、パッケージに入らないものがあることです。

- Package Managerで追加した機能
- タグ、レイヤー、入力設定

自分のプロジェクトにはこれらが全部そろっているのに、当然動きます。でも統合先は別のプロジェクトです。

- そこで、空のプロジェクトを作って、自分のパッケージだけを読み込んでみます。

ここで動けば、統合先でもほぼ確実に動きます。動かなければ、提出前に自分で直せます。

- UnityHubで「New project」を押して、新しいプロジェクトを作ります。

- テンプレート：自分のミニゲームと同じもの（2Dで作ったなら Universal 2D）
- Unityバージョン：チームで決めたバージョン
- プロジェクト名： `ImportTest` など、分かるもの

※バージョンが違くと、それだけで動かないことがあります。ここは必ずそろえてください。

- 新しいプロジェクトが開いたら、 `Assets > Import Package > Custom Package...` を選び、書き出したパッケージを読み込みます。

- 読み込んだら、この4つを確認します。

確認項目	見るところ
エラーが出ていないか	Consoleに赤いエラーが無い
シーンが開けるか	自分のシーンをダブルクリックして開く
見た目が崩れていないか	白い四角（Missing）になっている絵が無い
実際に遊べるか	Playボタンを押して動かしてみる

- 【よくあるエラー1】赤いエラーで「型または名前空間が見つかりません」

実際にやってみたところ、こういうエラーが出ました。

```
Assets\Scripts\Scene\GameManager.cs(1,13): error CS0234: The type or namespace name "VisualScripting" does not exist in the namespace "Unity" (are you missing an assembly reference?)
```

読み込み後に出たエラー

これは、スクリプトの先頭にある `using` が、統合先に無い機能を指しているときに出了。

```
using Unity.VisualScripting; // ← この機能が統合先に無い
using Unity.Mathematics;    // ← これも
```

対処は2つあります。

- その機能を実際に使っていない場合： `using` の行を消す（多くの場合こちらです。エディタが自動で追加しただけ）
- 実際に使っている場合： ReadMe.mdの「追加した機能」に書く

8. 【よくあるエラー-2】絵が白い四角になる

.metaファイルと一緒に書き出されていないか、書き出しのときに「Include dependencies」がオフだった可能性があります。

【4】の手順で、オンにして書き出し直してください。

9. 【よくあるエラー-3】タグ関連のエラーで止まる

`Tag: oo is not defined` のようなエラーが出たら、自分で追加したタグを使っています。

統合先に同じタグを追加してもらう必要があるので、ReadMe.mdの「追加したタグ」に必ず書いてください。

10. 動作確認ができれば、パッケージファイルを提出します。

提出前チェックリスト

- ☐ 自分のミニゲームのファイルが、1つのフォルダにまとまっている
- ☐ フォルダ名が `GameJam_<開始年yyyy>_<開始月mm>_Team<チーム名>_<ゲームタイトル>` になっている
- ☐ フォルダを移動したあと、Unityで再生して動いた
- ☐ ReadMe.md を書いて、フォルダの中に置いた
- ☐ ReadMe.md の想定プレイ時間を、15秒～5分の範囲で書いた
- ☐ 自分のフォルダを右クリックして書き出した（Assetsではない）
- ☐ 「Include dependencies」がオンの状態で書き出した
- ☐ ファイル名が決められた形式になっている
- ☐ ファイルサイズが極端に小さくない（数KBは中身が空の可能性）
- ☐ 空のプロジェクトに読み込んで、エラー無しで動いた

困ったとき逆引き表

こうなった	こうする
書き出したファイルが数KBしかない	フォルダを選ばずに書き出した可能性。自分のフォルダを右クリックしてやり直す
読み込んだら絵が白い四角になる	Include dependencies がオフだった。オンにして書き出し直す
型または名前空間が見つかりません エラー	統合先に無い機能を使っている。使っていないなら該当の <code>using</code> を消す。使っているなら ReadMe.mdに書く
<code>Tag: oo is not defined</code> エラー	自分で追加したタグを使っている。ReadMe.mdに書いて、統合先にも追加してもらう
Unityを開いたら Enter Safe Mode? が 出た	コンパイルエラーが残っている。Ignoreで開いて、Consoleの赤いエラーを直す

フォルダを移動したら参照が外れた	エクスプローラーで移動した可能性。Unity上で移動し直す
ReadMe.md のファイル名がゲーム名になっている	HackMDのダウンロードはメモ名がファイル名になる。 <code>ReadMe.md</code> に変更する

【6】迷とら運営：統合プロジェクトに読み込む

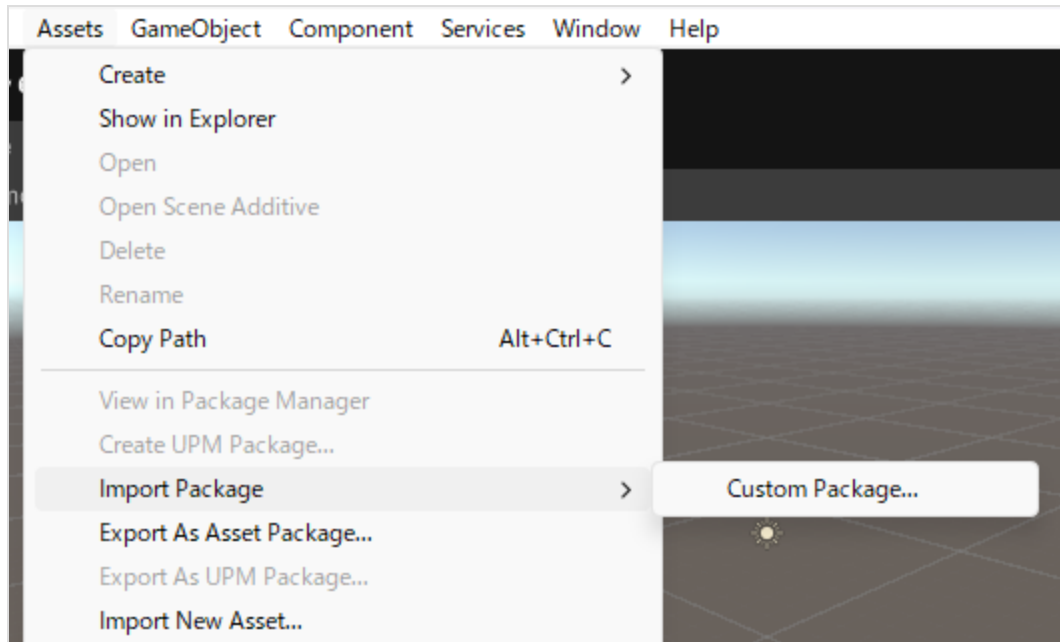
1. ここからは受け取る側、迷とら運営の作業です。

提出する側も目を通しておくと、自分の提出物がどう扱われるかが分かります。

2. 全チームのパッケージがそろったら、統合プロジェクトを開きます。

まだ無ければ、UnityHubで新規プロジェクトを作ります（テンプレートとバージョンは、チームで決めたものにそろえます）。

3. Assets > Import Package > Custom Package... を選びます。

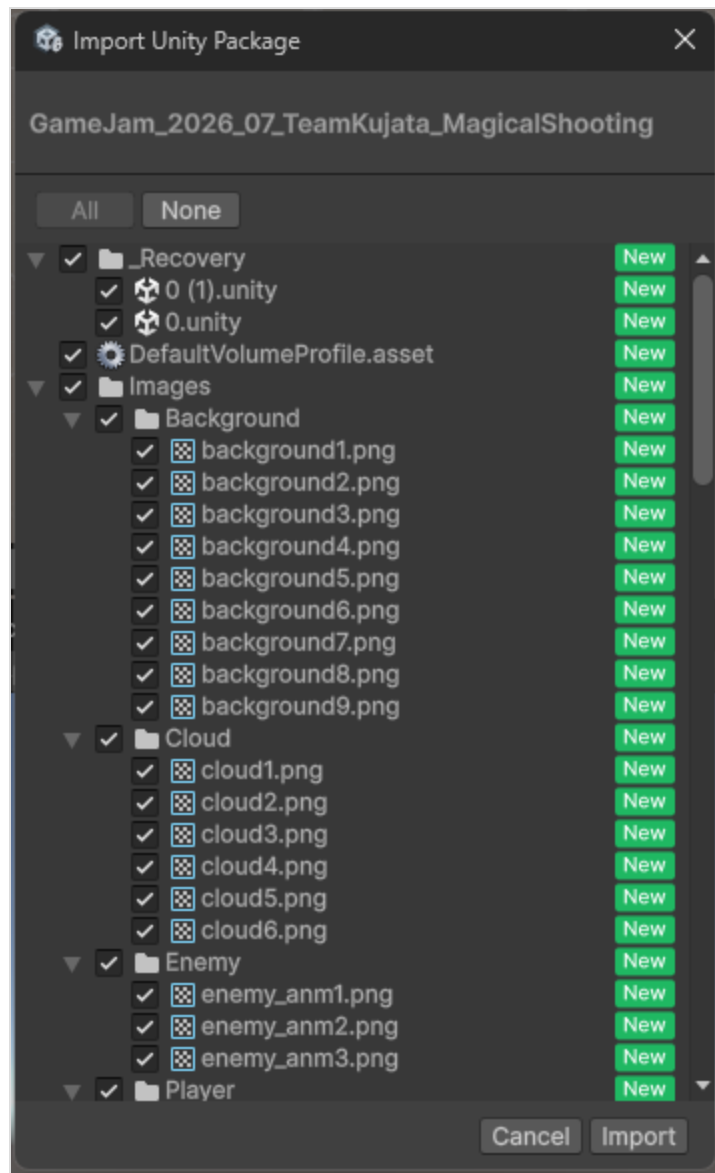


Import Packageメニュー

4. ファイル選択の画面が出るので、1チーム分のパッケージを選びます。

必ず1つずつ読み込みます。まとめて読み込むと、問題が起きたときにどのチームのものが原因か分からなくなります。

5. 「Import Unity Package」というウィンドウが開きます。中身の一覧が表示されるので、そのまま右下の「Import」を押します。

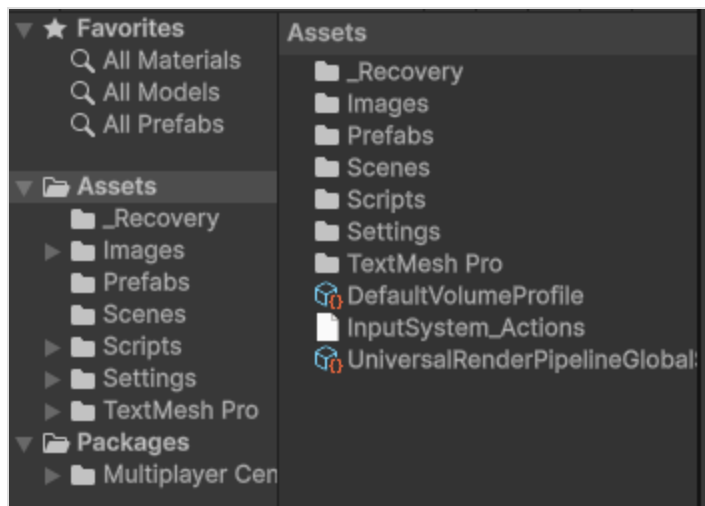


Import Unity Packageウィンドウ

- 上部にパッケージ名が表示されるので、どのチームのものか確認できます
- 緑の「New」は新しく追加されるファイル、「Changed」は既にあるファイルが上書きされるという意味です

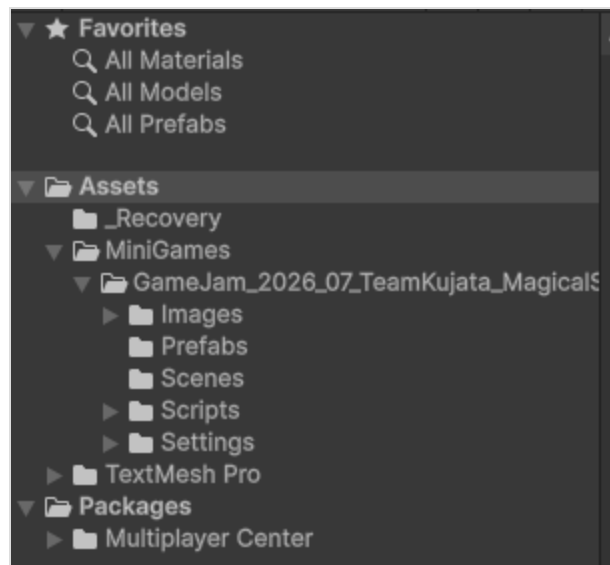
「Changed」が出た場合は注意が必要です。他のチームのファイルを上書きしようとしている可能性があるため、Importを押さずに提出したチームへ確認します。

6. 読み込みが終わると、Projectウィンドウにファイルが追加されます。



読み込み後のProjectウィンドウ

各チームがフォルダをまとめているので、チームごとに分かれた状態になります。



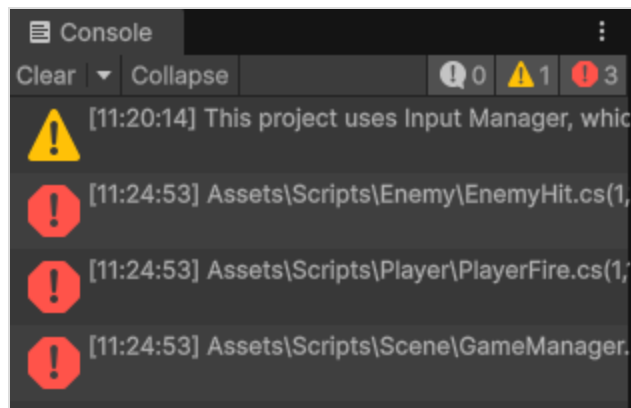
チームごとに分かれた状態

7. 読み込んだら、まずそのチームのReadMe.mdを開きます。

- 追加した機能 → 統合プロジェクトにも同じものを入れる
- 追加したタグ・レイヤー → `Edit > Project Settings > Tags and Layers` で追加する
- 最初に開くシーン → タイトル画面からつなぐ先として控える
- 想定プレイ時間 → ミニゲームの制限時間として設定する

これを先にやっておくと、次のエラー確認がスムーズになります。

8. 次にConsoleを確認します。赤いエラーが出ていたら、そのチームのミニゲームはまだ動きません。



読み込み後に出たエラー

実際に試したところ、3つのエラーが出ました。内容はこれです。

Assets\Scripts\Scene\GameManager.cs(1,13): error CS0234: The type or namespace name 'VisualScripting' does not exist in the namespace 'Unity' (are you missing an assembly reference?)

エラーの詳細

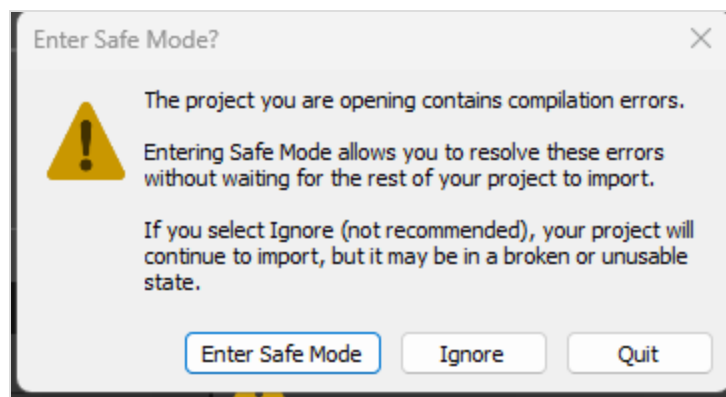
error CS0234: The type or namespace name 'VisualScripting' does not exist in the namespace 'Unity'

「スクリプトが使っている機能が、このプロジェクトに入っていない」という意味です。パッケージに入らないものが原因です。

9. 対処は2つあります。

- Window > Package Management > Package Manager から、エラーに出ている機能を追加する
- その機能を実際には使っていない場合、スクリプトの `using` の行を消す（提出したチームに確認してから）

10. コンパイルエラーが残った状態でUnityを開き直すと、この画面が出ます。



Enter Safe Mode?ダイアログ

- **Enter Safe Mode**：エラーを直すことに集中できるモードで開く
- **Ignore**：とりあえず無視して開く（エラーは残ったまま）

どちらを選んでも、エラーを直さない限りゲームは動きません。エラーは読み込んだその日に直します。

11. 全チーム分は、この順番で進めます。

1チーム読み込む → ReadMe.mdの内容を反映 → エラーを確認 → 直す → 動作確認 → 次のチームを読み込む

まとめて読み込むと、エラーが混ざって原因が分からなくなります。

12. 最後に、各ミニゲームのシーンをBuild Settingsに登録します。

読み込んだだけではビルドに含まれないため、`File > Build Profiles` を開いて、Scene Listに全ミニゲームのシーンを追加します。

ここでReadMe.mdの「最初に開くシーン」を使います。